Slides: Restricted access to CS 422 Fall 2026; not to be shared online or in public forums

Slides adapted from Prof. Stefan Schmid's course at TU Berlin, and the Top-Down Approach textbook by Kurose-Ross

Transport Layer

CS 422: Computer Networks

Instructor: Vamsi Addanki

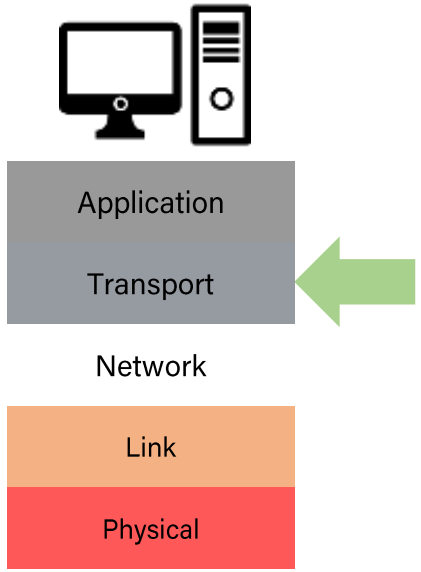
TAs: Youngsuk Kim, Xiao Luo, Albert Vo

04 September 2026

PURDUE UNIVERSITY | Department of Computer Science

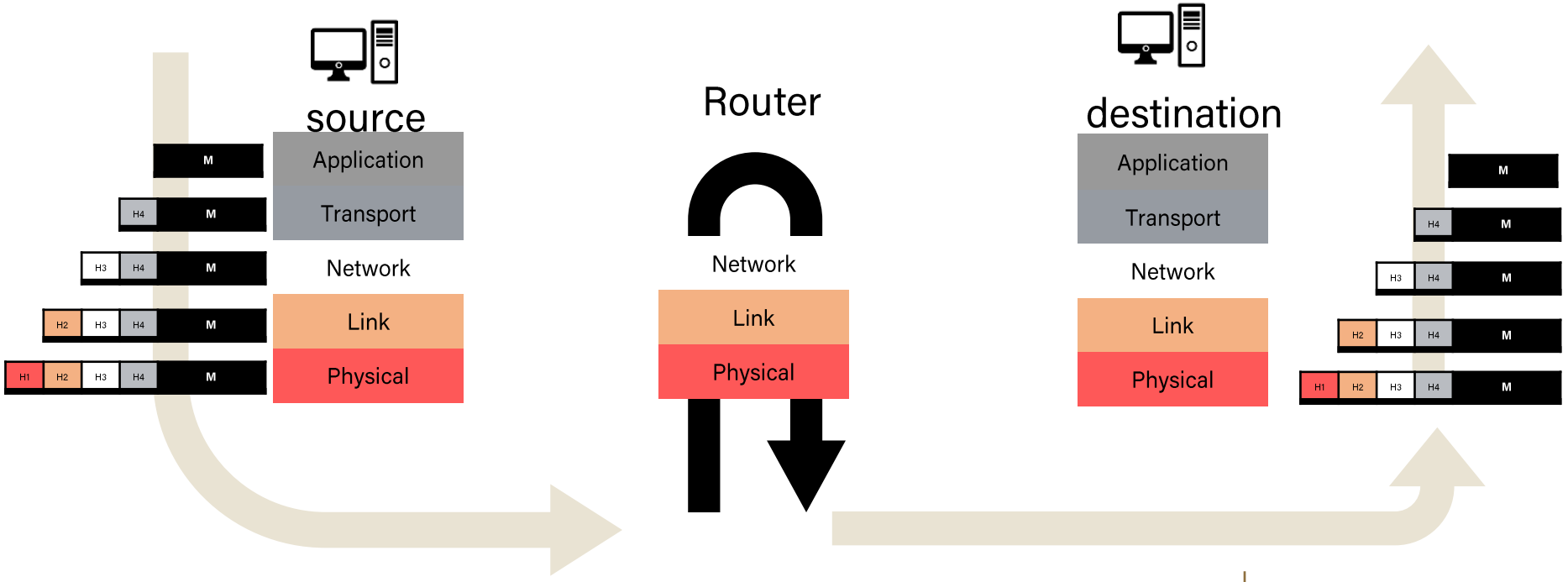
Reminder

- Transport layer for communication between applications
  - Over the network
  - And on same host
- Builds on network layer to provide service
- “Ports” used for addressing



Reminder

- End-to-end communication between host on the edge of the network
- The main transport protocols: TCP and UDP



Sockets: Interface to Applications

Socket API

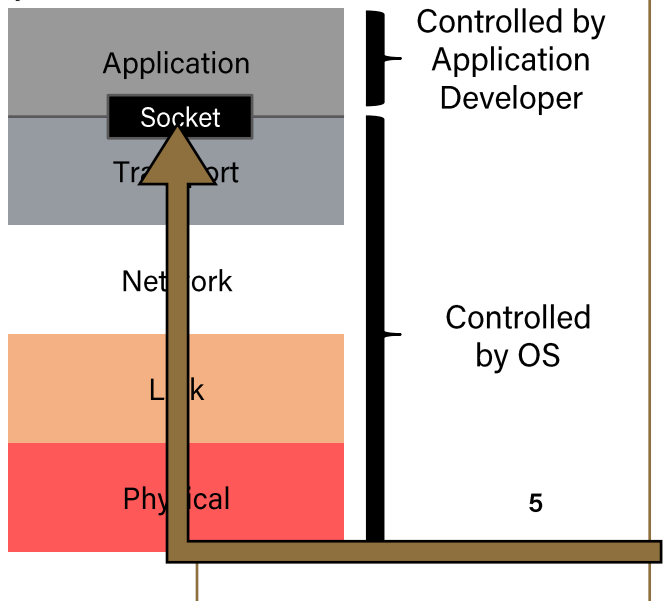
- Introduced in BSD4.1 UNIX, 1981
- Explicitly created, used, released by apps
- Client/server paradigm
- Two types of transport service via socket API:
  - Unreliable datagram
  - Reliable, byte stream-oriented

socket

A *host-local*, *application-created/owned*, *OS-controlled* interface (a "door") into which application process can both send and receive messages to/from another (remote or local) application process

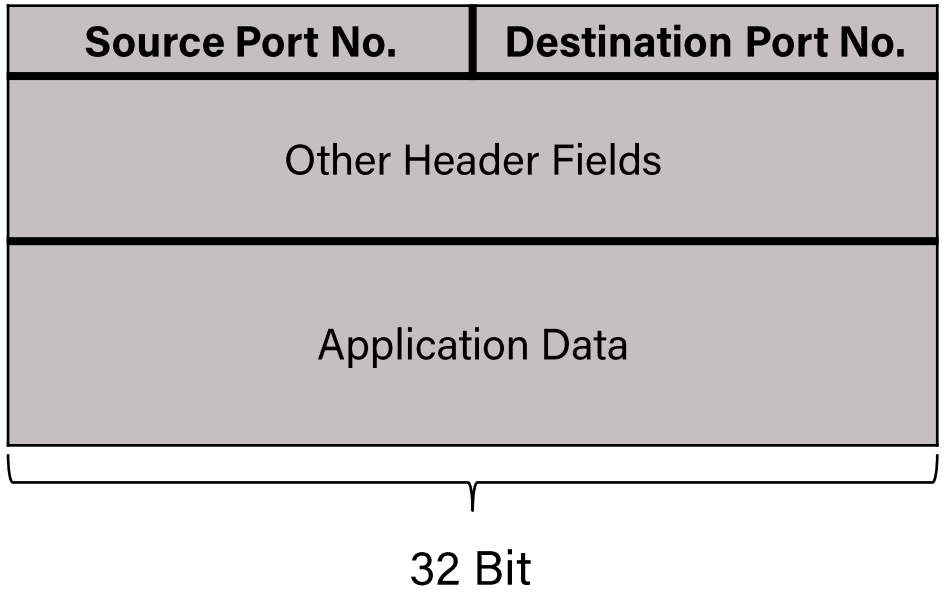
Sockets and OS

- Sockets: A “door” between application process and end-end-transport protocol (UDP or TCP)
  - Allows an application to send and receive messages to/from another application
  - These interfaces can be local (on same machine) or remote (on different machine)
  - Owned and created by the application (application “opens” sockets)
  - One application can have multiple open sockets
  - Functionality provided by the operating system (OS)



Port Numbers

- More precisely: Transport layer communication is between sockets
- How to address a single sockets on a host?
  - Port numbers in the header of the transport layer
  - Some ports are reserved for specific applications (e.g. port 80 is for HTTP)



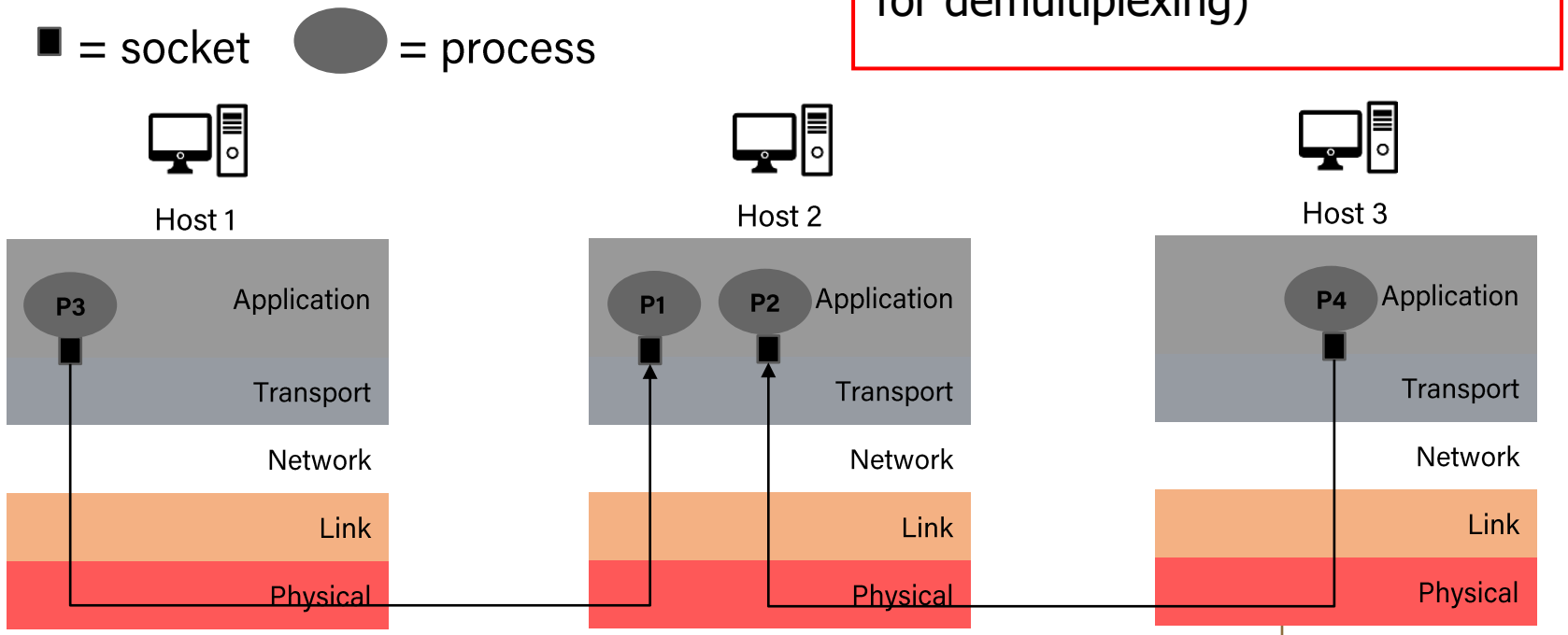
Transport-layer (De-)Multiplexing

Demultiplexing at rcv host:

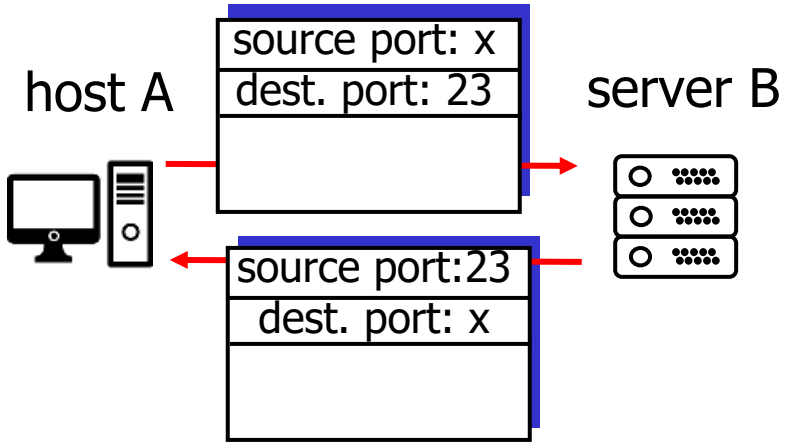
Delivering received segments to correct application (socket)

Multiplexing at send host:

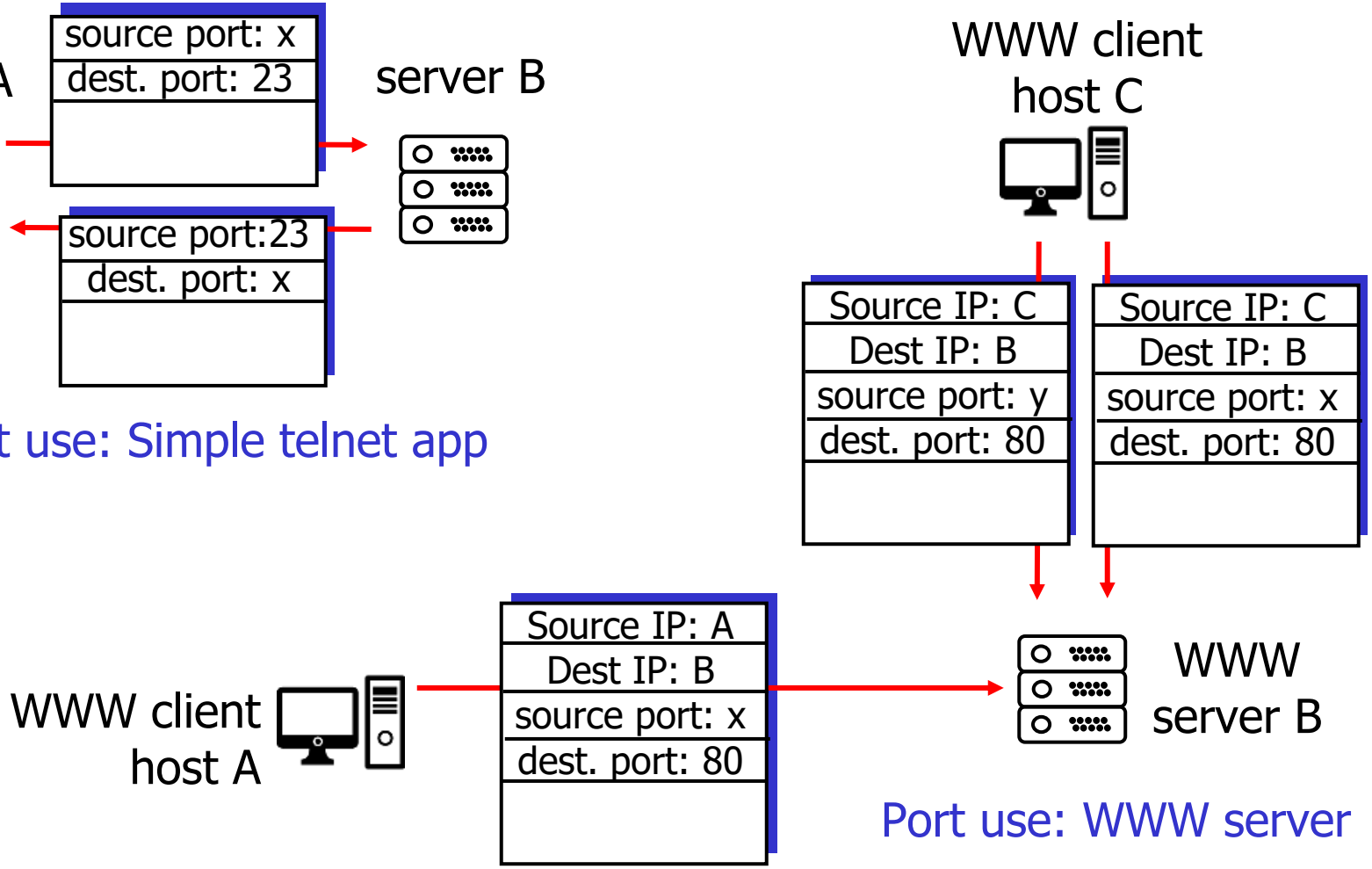
Gathering data from multiple appl. (sockets), enveloping data with header (later used for demultiplexing)



Multiplexing / De-Multiplexing: Examples



Port use: Simple telnet app



Port use: WWW server

8

UDP: User Datagram Protocol RFC 768

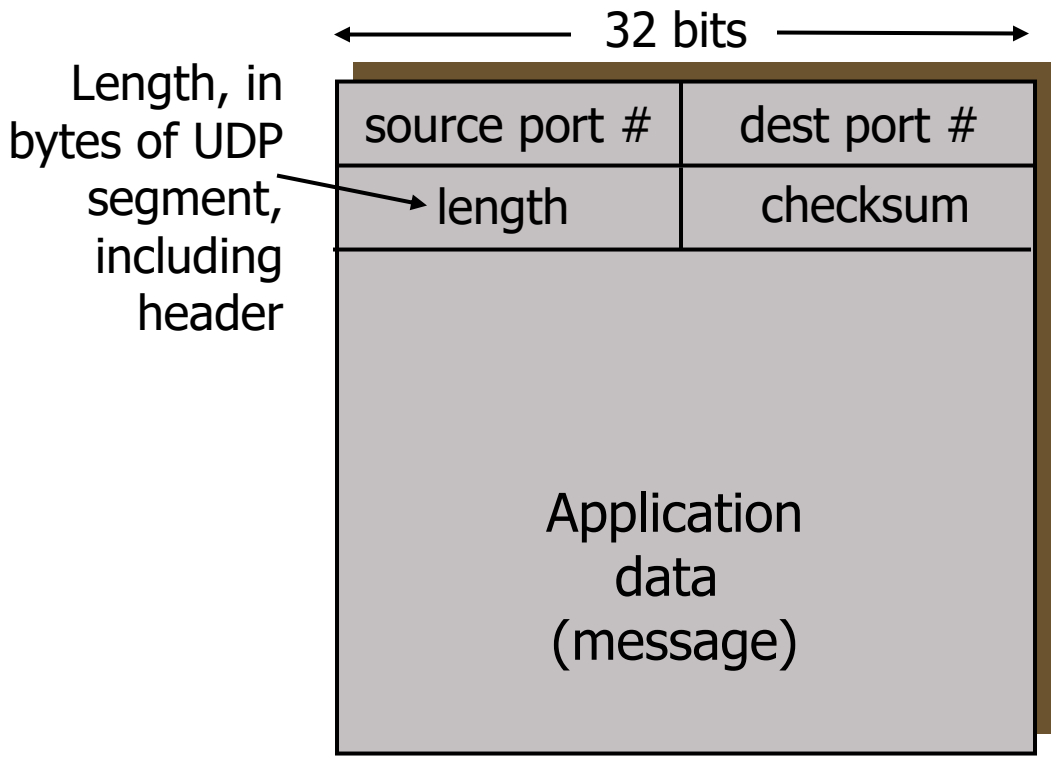
- "Bare bones" Internet transport protocol
- "Best effort" service, UDP segments may be:
  - Lost
  - Delivered out of order to application
- *Connectionless*:
  - No handshaking between UDP sender, receiver
  - Each UDP segment handled independently of others
  - Receive buffer exists, but no sender buffer

Why is there a UDP?

- No connection establishment (which can add delay)
- Simple: no connection state at sender, receiver
- Small segment header
- No congestion control: UDP can blast away as fast as desired

UDP: Usage and Segment Structure

- Each user request transferred in a single datagram
- UDP has a receive buffer but no sender buffer
- Often used for streaming multimedia apps
  - Loss tolerant
  - Rate sensitive
- Other UDP uses (why?):
  - DNS, SNMP, NFS
- Reliable transfer over UDP: add reliability at application layer



UDP segment format

UDP Usage

- Which applications use UDP?
  - Multimedia streaming
    - Loss tolerant
    - Rate sensitive
  - DNS: Fast because no connection is necessary
- Sometimes blocked for security reasons

UDP Checksum

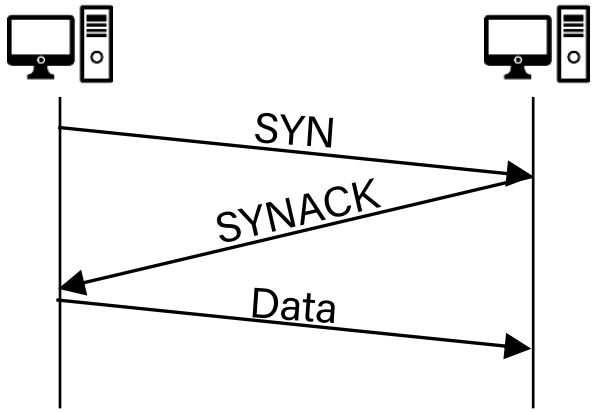
- Ones-complement of 16 bit words
  - Overhead is attributed for with a wrap around
  - Receiver will add all 16bit words with wraparound and the checksum, if result is vector of 1s → No error occurred
- Covers data plus a 12 byte pseudo header
  - IP addresses, protocol identifier, length
  - Ensures that packet has reached the correct host
- Pad byte in case of an odd packet length
- Optional: Checksum=0 indicates no checksum
  - Should always be enabled

Advantages/Disadvantages of UDP

- Disadvantages
  - Messages might get lost
  - Messages might arrive out of order
  - Sender might overwhelm receiver with messages
- Advantages
  - Simple, no state for sender or receiver necessary
  - Small transport layer header
  - No additional delay for connection establishment
  - UDP can use up all the available bandwidth (because there is no congestion control)
  - Allows to communicate with multiple receivers over one socket
  - Application has more control over what data is sent and when

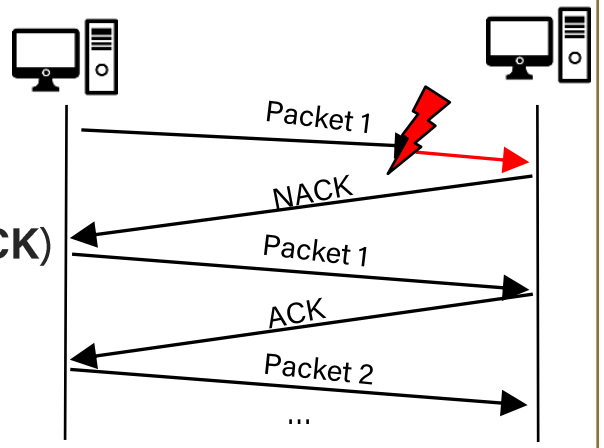
How to Build a Reliable Protocol

- Not all applications tolerate packet loss well
  - A protocol needed that can establish a reliable channel
  - Connection oriented protocol!
  - We take a step-by-step approach …
- Step 1: Reliable transfer over a reliable channel
  - All packets will arrive always and in correct order
  - We need to make sure the receiver **is ready to receive**



How to Build a Reliable Protocol

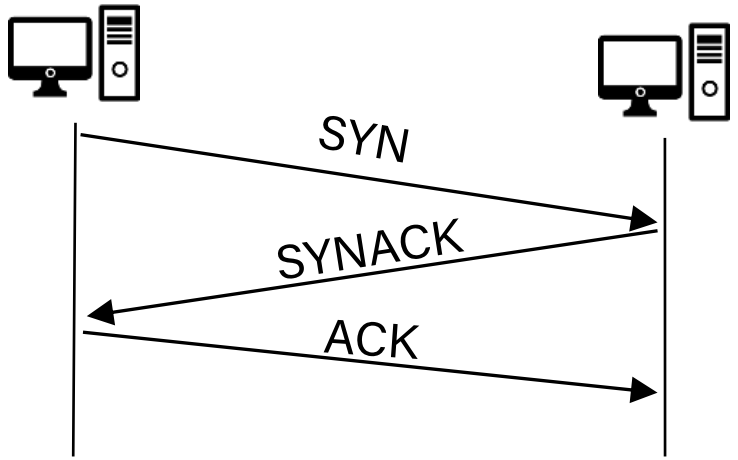
- Step 2: Reliable transfer over channel with bit errors
  - Data can be corrupted, but in right order
  - We need to check if arrived intact
    - Send data in **packets**
    - Positive acknowledgement when packet is intact (**ACK**)
    - Negative acknowledgement when not intact (**NACK**)
    - **Repeat corrupted packets**



How to Build a Reliable Protocol

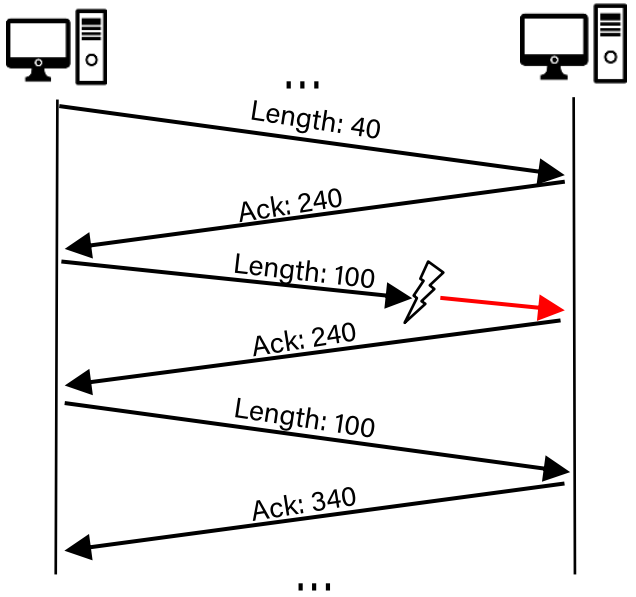
- SYNACK on establishing connection could be corrupted
  - SYNACK to NACK
  - Receiver of corrupted message closes connection
  - Sender of corrupted message thinks connection is established

➔ Additional ACK to indicate that SYNACK has been received



How to Build a Reliable Protocol

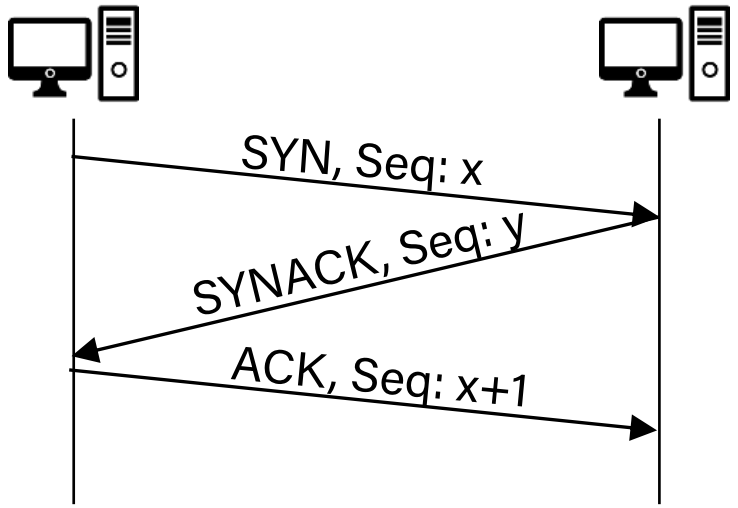
- Step 2: Reliable transfer over channel with bit errors (Cont.)
  - NACK can also be corrupted
  - ACK: send No. of bytes received since connection was established
  - NACK: Repeat No. of last byte read successfully before this packet (“**duplicate ACK**”)



How to Build a Reliable Protocol

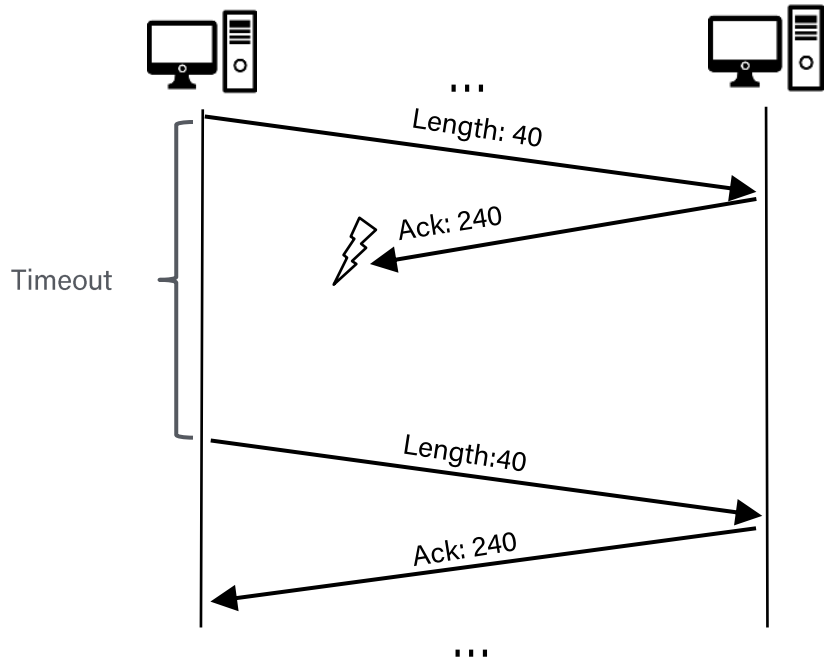
- Step 3: Reliable transfer over an unreliable channel
  - Packets can be lost or arrive out of order

    ➔**Sequence numbers**
  - Each packet has a sequence number
  - Both sides increase the sequence number independently



How to Build a Reliable Protocol

- Step 3: Reliable transfer over an unreliable channel (Cont.)
  - How to detect packets that were lost?
  - No ACK or duplicate ACK for long time: **Timeouts**



TCP

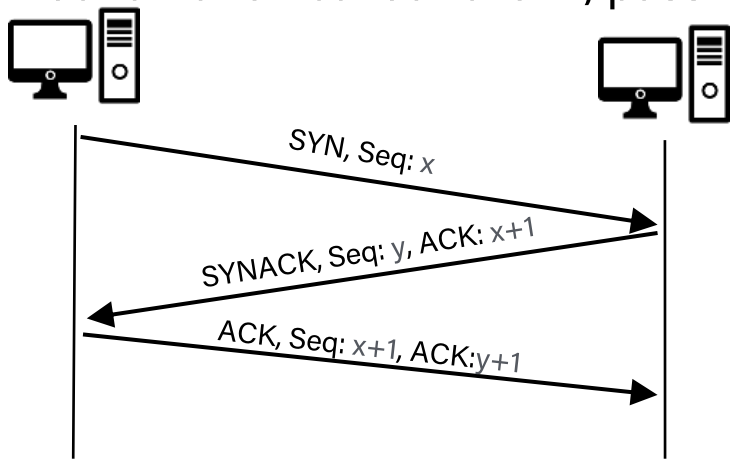
- Transport protocol for in-order reliable data transfer between applications
- Point-to-point communication
  - One sender, one receiver
  - For each tuple of (source IP, dest. IP, source port, dest. port) a new socket is opened
- TCP takes a **stream of bytes** from the user
  - Segment stream into chucks
  - Reassemble on receiver side
  - No “message boundaries”
- Full duplex
  - Allows communication in both directions (unlike UDP)

TCP

- Connection oriented
  - Connections are established and terminated
  - Handshaking to exchange control messages
  - Protocol has to remember state of connection
- Flow Control
  - Sender will not overwhelm receiver
- Congestion Control
  - Sender will not overwhelm network

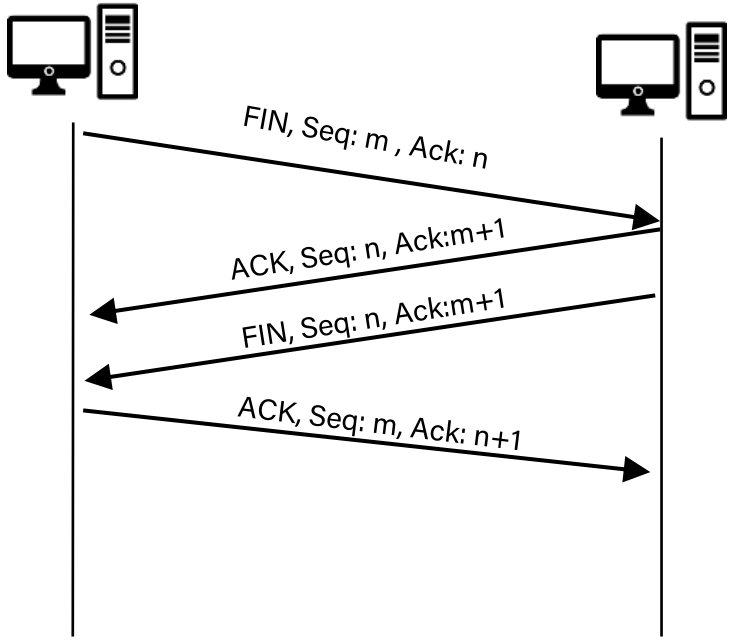
Establishing a TCP Connections

- "3-Way-Handshake"
- SYN, SYNACK and ACKs are flags in the TCP header
- Both parties select their initial sequence number independently
- SYNACK and ACK packets increment last sequence number of other side and use it as ACK number
- Initiator of connection often called "client", passive side is often called "server"

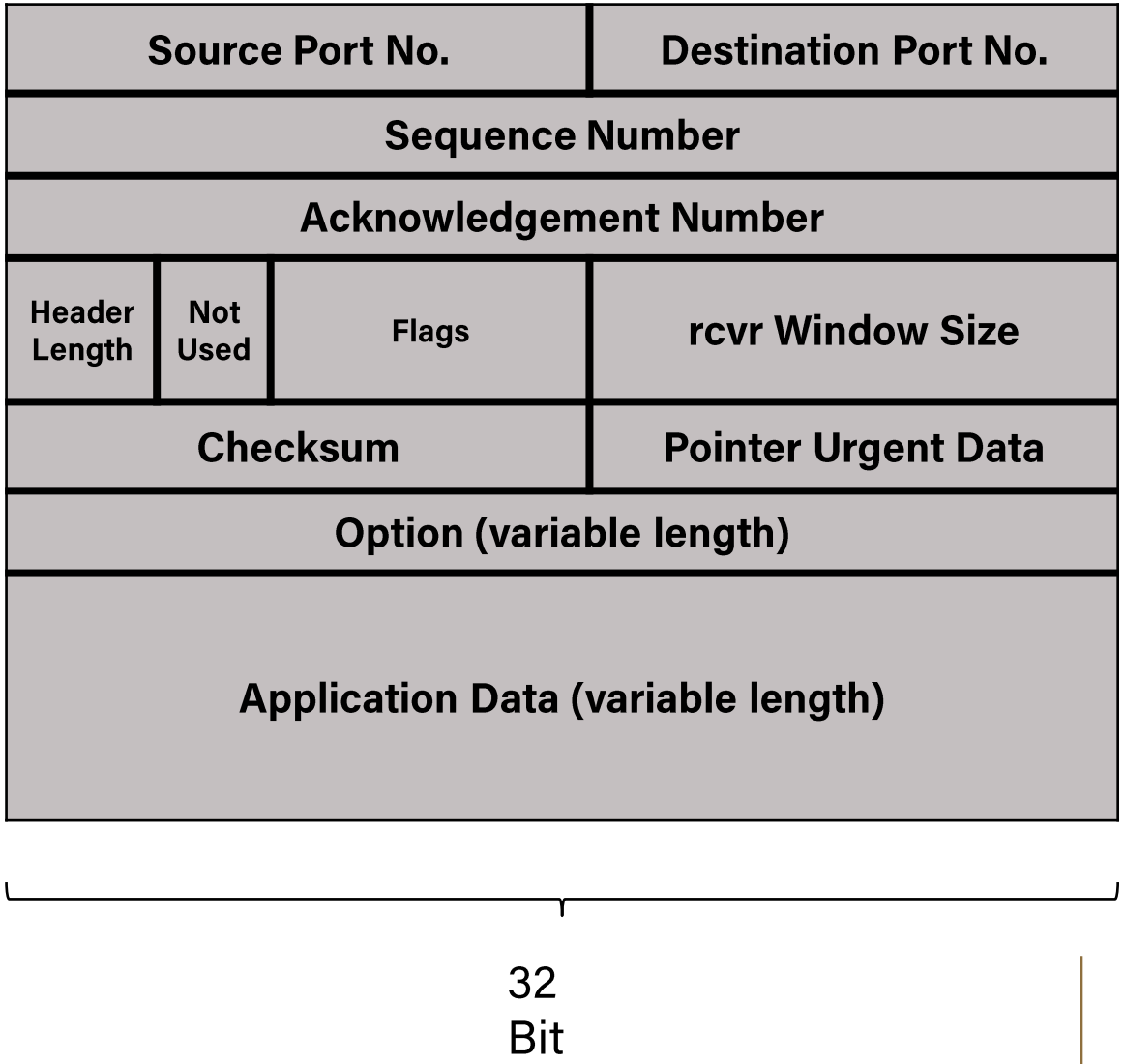


Terminating a TCP Connection

- Receiver side of connection has to know, that no packets will be transferred anymore
- FIN indicates, that this side has no more packets to sent



TCP Segment Structure

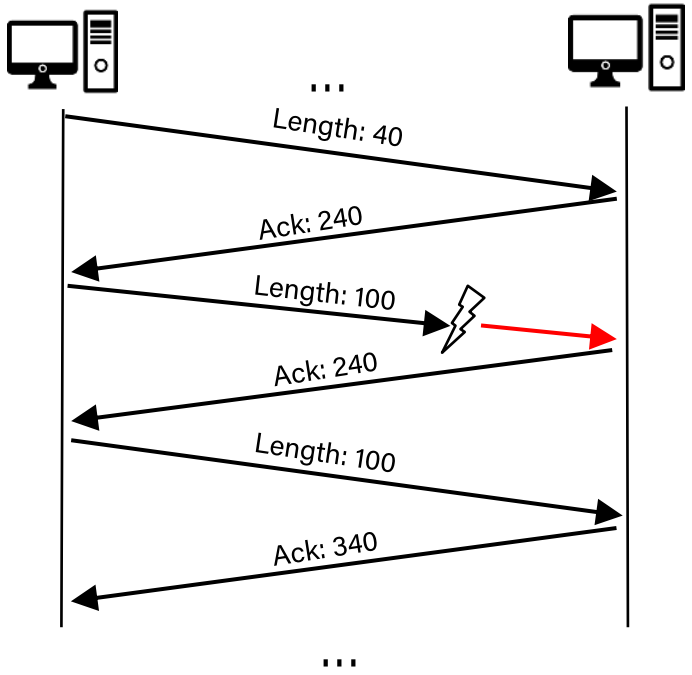


TCP Segment Structure

- Sequence number and ACK number counted by bytes of data (not segments!)
- Flags in TCP Header (each 1 Bit long)
  - URG: Urgent data (generally not used)
  - ACK: ACK number valid
  - PSH: Push data now
  - SYN: Establishing connection
  - RST: Connection reset
  - FIN: Closing connection
- Checksum similar to UDP
- Rcvr Window Size: Number of bytes receiver is willing to accept

ACKs

- ACKs are always cumulative in TCP
- When ACK received: All data before the byte with number from ACK is considered successfully transmitted
- Repeating old ACK will cause retransmission



TCP ACK Generation

Event at Receiver	TCP Receiver action
Arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	Delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK (reduces ACK traffic)
Arrival of in-order segment with expected seq #. One other segment has ACK pending	Immediately send single cumulative ACK, ACKing both in-order segments
Arrival of out-of-order segment higher-than-expect seq. # . Gap detected	Immediately send *duplicate ACK*, indicating seq. # of next expected byte (trigger fast retransmit)
Arrival of segment that partially or completely fills gap	Immediate send ACK, provided that segment starts at lower end of gap

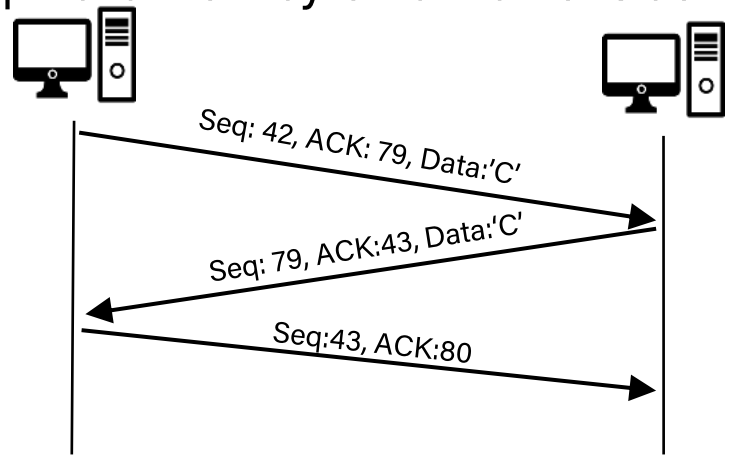
Delayed and Piggybacked ACKs

- ACK messages
  - No content
  - Only ACK flag is set

  ➔ Large overhead
- Delayed ACK
  - Wait up to 500ms to sent ACK
  - In case another segment arrives in that time: Combine ACK for both segments in one message
- Piggybacked ACK
  - ACK messages is combined with message containing other Data
  - E.g. very common to combine FIN and ACK when closing connection

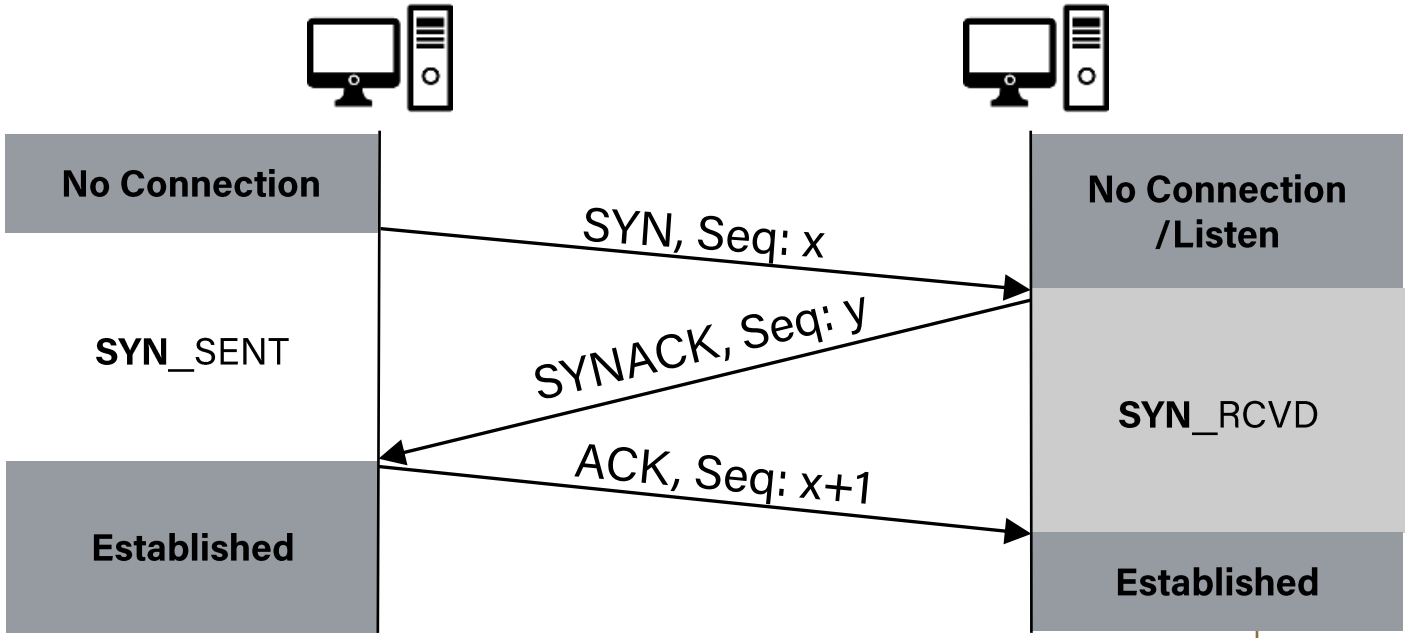
TCP Sequence Numbers

- Mechanism to detect out-of-order packets
- On handshake: Each side chooses own initial sequence number
- Why not start sequence numbers on one?
  - Packets from other connection could be mistaken
  - Adversary could guess sequence numbers to insert packets
- Seq. No. is number of first byte in this data segment
- ACK No. is Seq. No. of next byte from other side

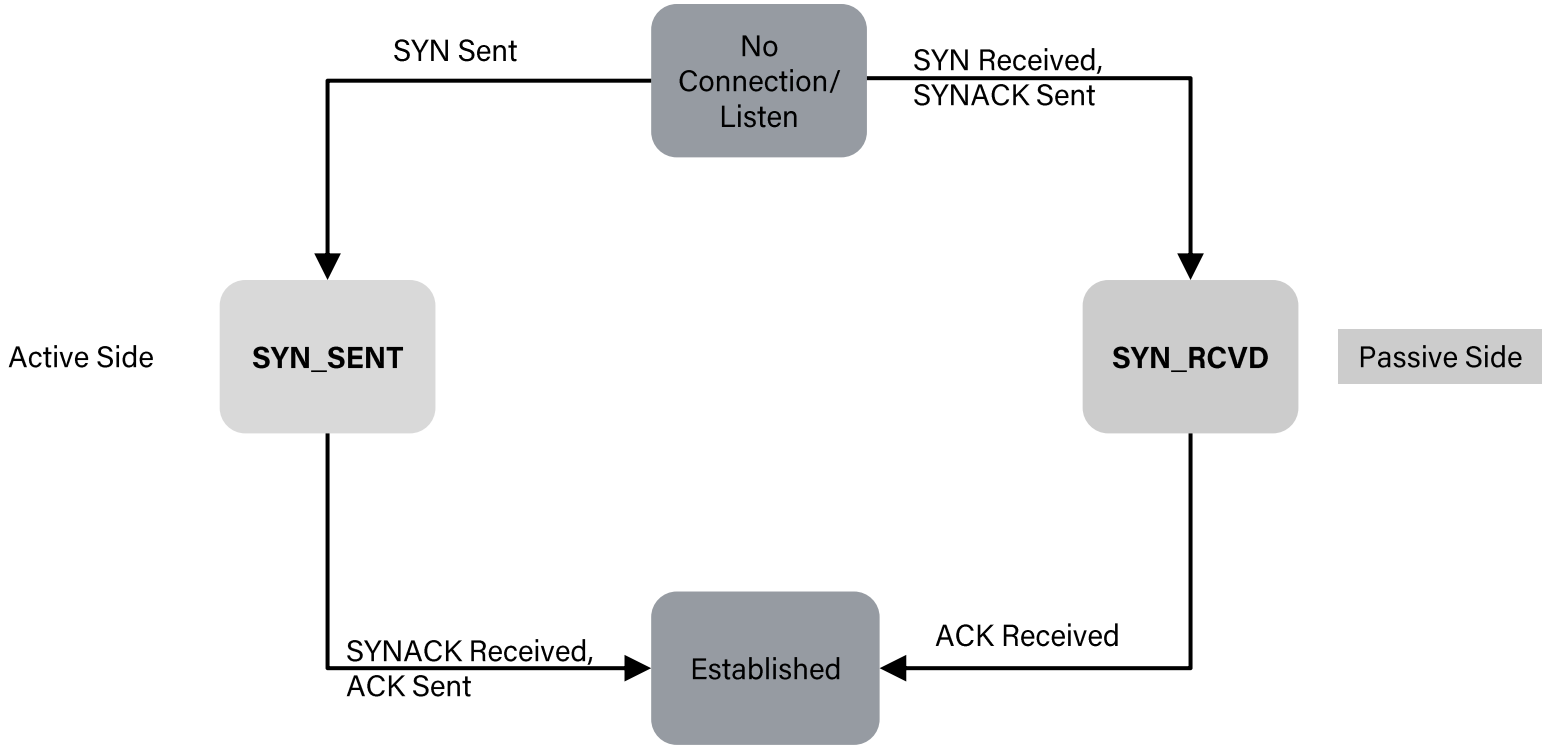


TCP and State

- Participants of a TCP connection have to keep a state
- State transition representable as state automata
  - SYN, SYNACK, FIN… change the state

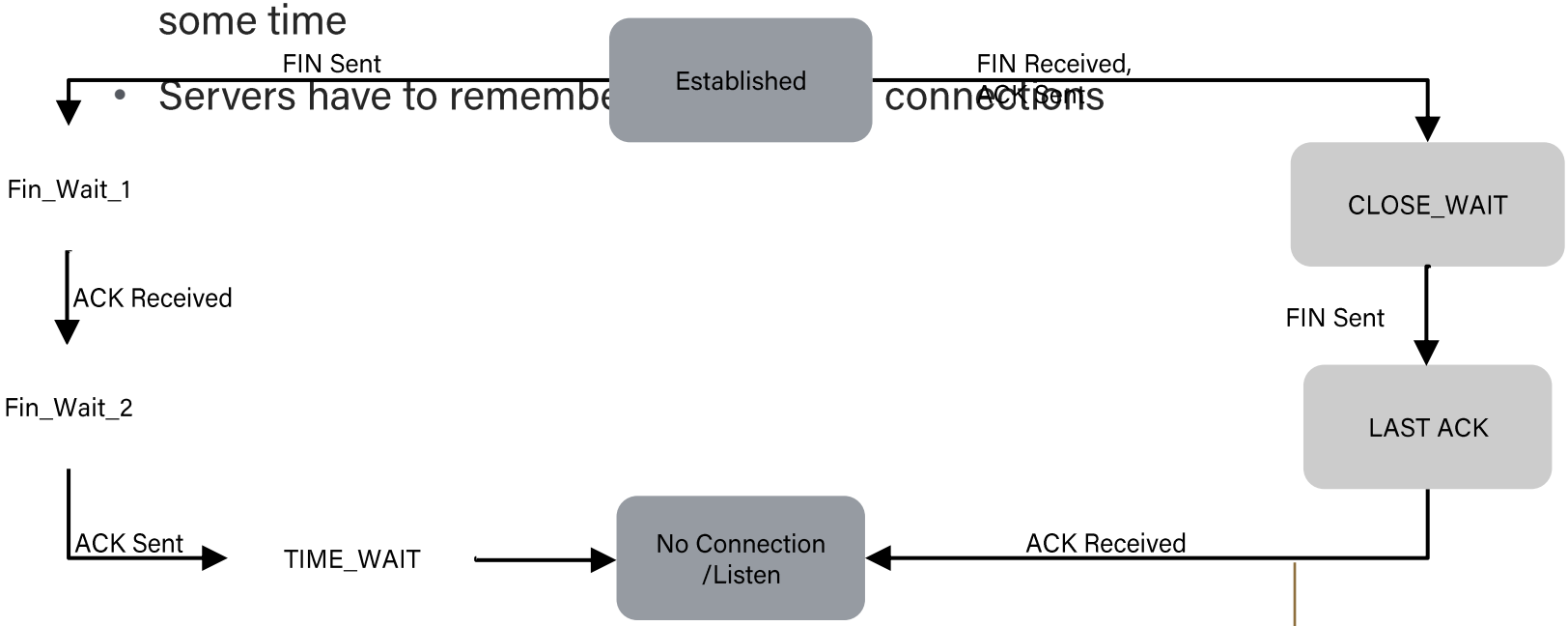


State Machine: Establishing a Connection



State Machine: Closing a Connection

- Why is there a TIME_WAIT state? Why is that a problem?
  - Last ACK to client could be lost (resending necessary)
  - Generally server closes actively, so server has to remember connection for some time
  - Servers have to remembe connections



Simple State Machine

- Resets, transmission errors, unexpected messages… ignored in these two automata
- RESET message
  - Sent when packets are received that do not belong to any known connection
  - Any data that has not been acked yet will be considered lost, when a RESET is received
  - Necessary e.g. when one side crashed and forgot all connections
- Actual state automata much more complicated!

TCP State Machine

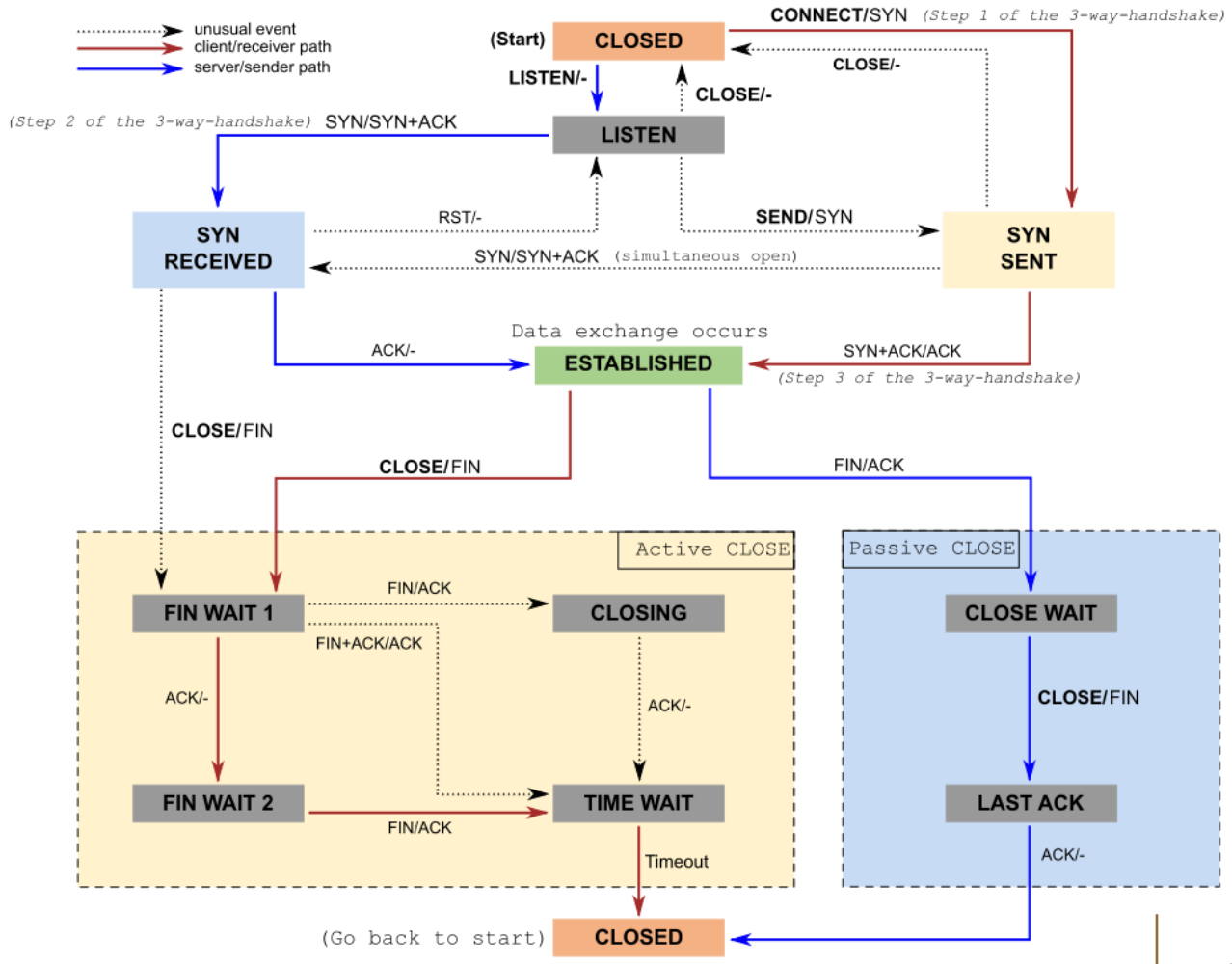
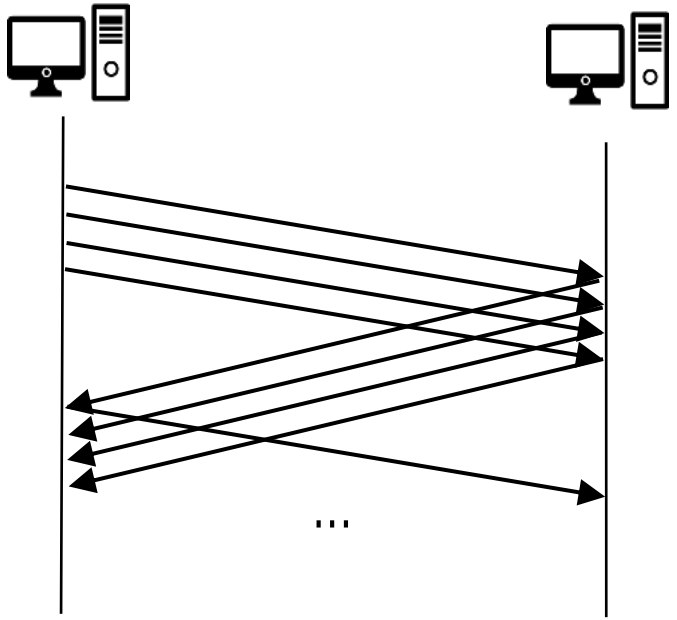


Image: Scil100, Wikipedia.org

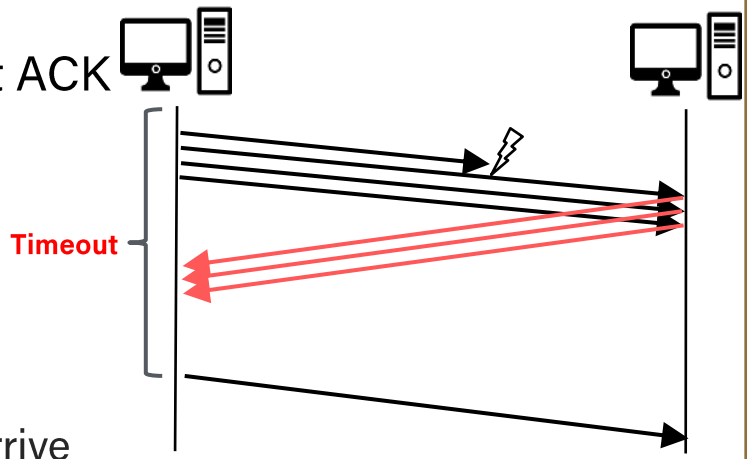
TCP Pipelining

- Low throughput if protocol is stop-and-wait
- Line would be mostly idle
- Therefore: Send out packets, before corresponding ACK has arrived
- So sender does not get overwhelmed: Flow Control
- So network does not get overwhelmed: Congestion Control



Retransmission

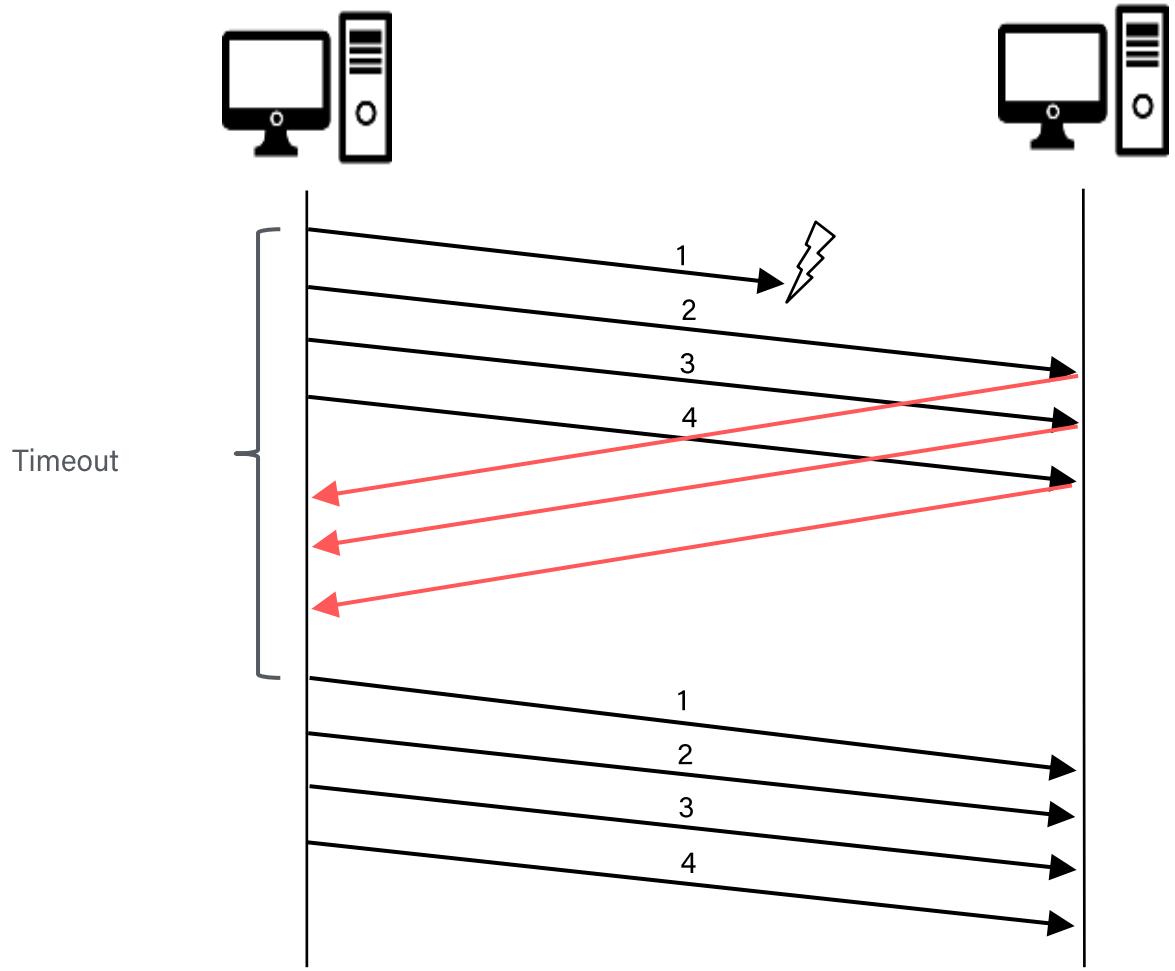
- In case packet is lost: Retransmission of last ACK
- How to detect that packet is lost
  - **Retransmission timeout** (RTO, as seen earlier)
  - **Triple duplicate ACK**
    - When packet was lost, old ACK is repeated
    - Due to pipelining: Multiple duplicate ACKs arrive
- Timeout can be very long, so throughput decreased when packets are lost often



Go-Back-n

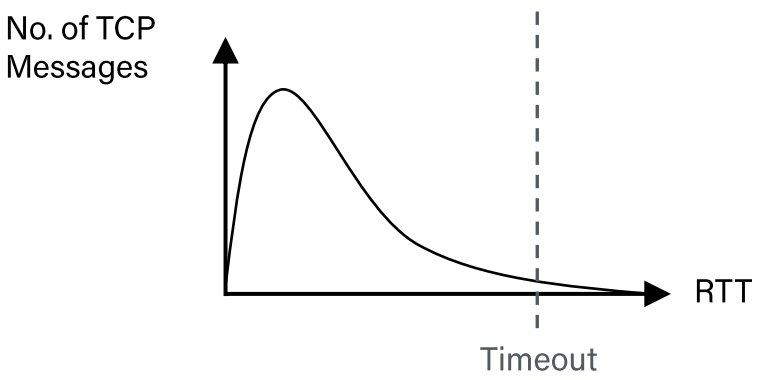
- Sending an ACK in TCP:
  - All bytes up to this point have been received successfully (cumulative ACK)
- Sender Side
  - All Errors are detected with timeouts
  - In case of error: All packets starting from corrupted packet are repeated
- Receiver Side
  - Sends ACK when receiving next Packet in order
  - Out-of-order packets are discarded
  - (Some variants buffer these packets to increase efficiency)

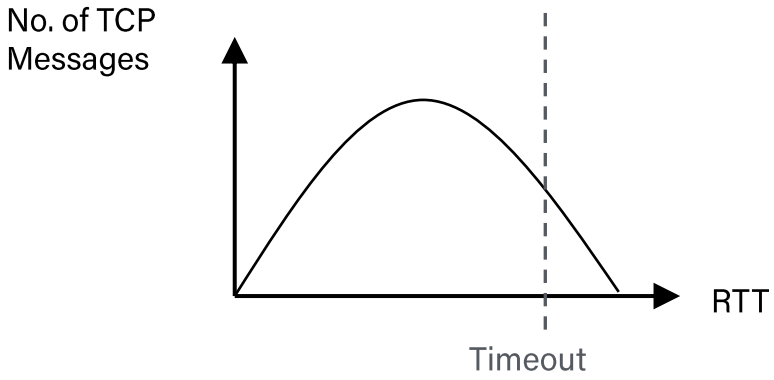
Go-Back-n



RTT Estimation

- RTT can vary
- Long timeout causes decrease in throughput, because of wait time before retransmissions
- How do we know the timeout is not too short or too long?





RTT Estimation

- Sender measures RTT and takes samples
- One sample: Time between sending segment and arrival of corresponding ACK
- Estimated RTT RTT_{est} is consciously updated

$$RTT_{est,new} = (1 - \alpha) \times RTT_{est,old} + \alpha \times SampleRTT$$

  - $\alpha = \frac{1}{8}$ for most TCPs
  - "exponentially weighted moving average"
- Observation: Large amounts of traffic in network cause highly variant RTTs

➔Take Variance of RTT into account

  - $\Delta_{RTT} = |SampleRTT - RTT_{est,old}|$

$$DevRTT_{new} = (1 - \beta) \times DevRTT_{old} + \beta \times \Delta_{RTT}$$

  - $\beta = \frac{1}{4}$

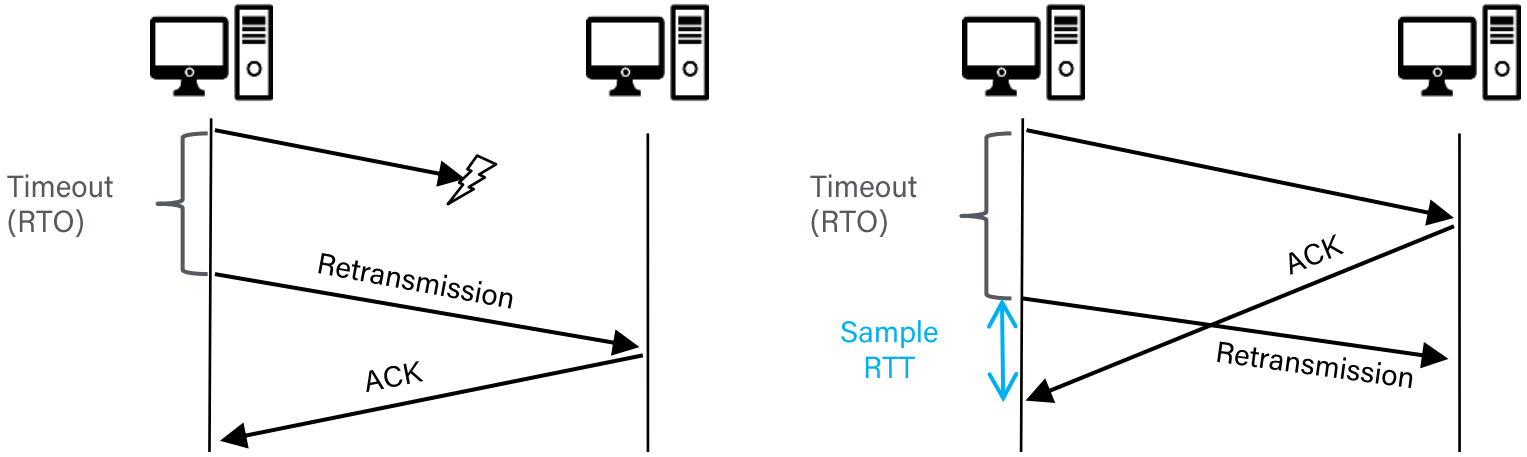
Setting RTO

- $RTO > RTT_{est}$, so no unnecessary retransmits
- But not too large, retransmits take very long
- RTO set to allow fluctuations according to current state of the network

$$RTO = RTT_{est} + 4 \times DevRTT_{old}$$

- Every time timeout occurs, RTO increases exponentially

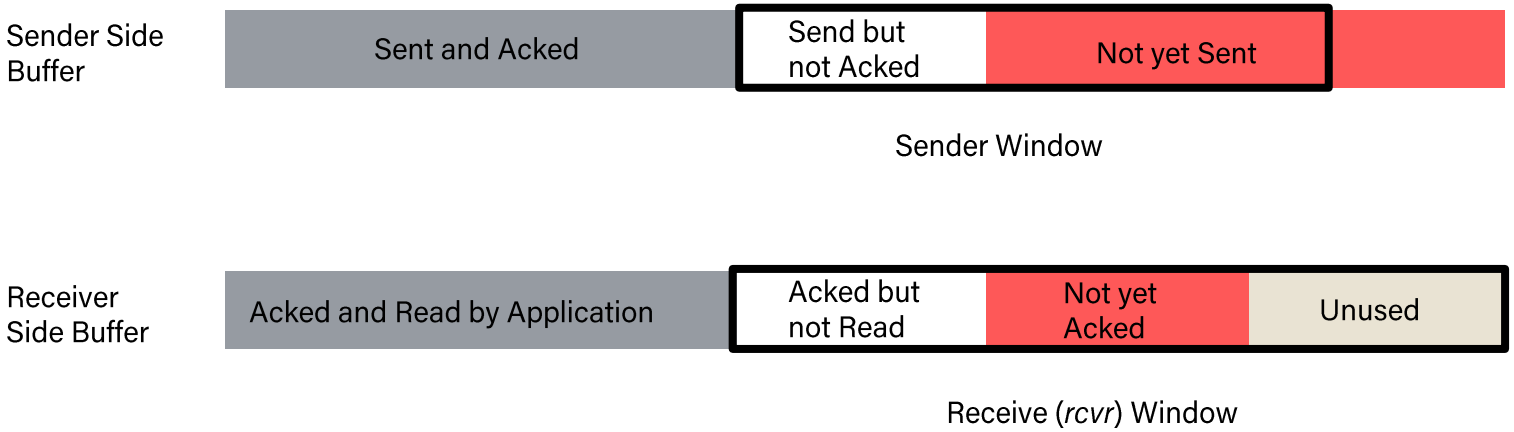
Retransmission Ambiguity



- Karn's RTT Estimator
  - If a segment has been retransmitted
  - Don't count RTT sample on ACKs for this segment
  - Keep backed off time-out for next packet
  - Reuse RTT estimate only after one successful transmission

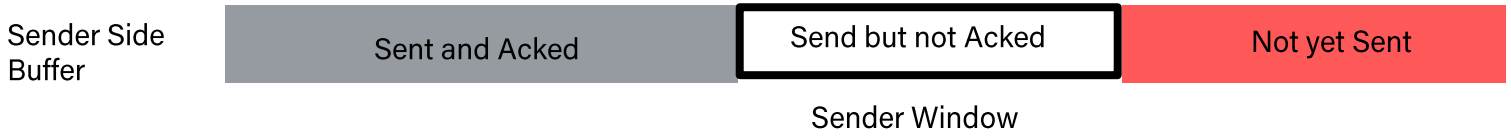
Flow Control

- Each side of a connection has buffers for sending and receiving
- TCP has to ensure that all packets are somehow received
  - Receiving side has to have space in buffer
- Flow Control: Ensures sender won't overrun receiver's buffers by transmitting too much, too fast

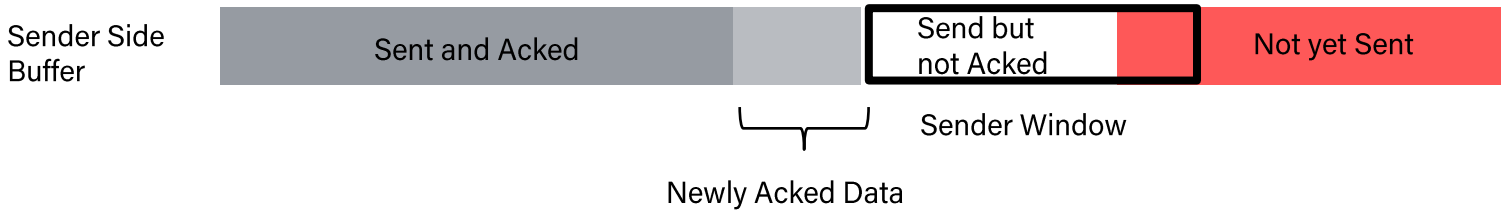


Flow Control: Windowing Mechanism

- Window is closed:



- New Ack arrives and window moves:



Flow Control

- TCP is a sliding window protocol
  - For window size n, can send up to n bytes without receiving an acknowledgement
  - When the data is acknowledged, the window slides forward
- TCP is **self-clocking!**
  - New data sent when old data is acked
  - Ensures an “equilibrium”
- Original TCP always sent entire window
  - Congestion control now limits this via congestion window determined by the sender! (**network limited**)
  - If not, data rate is **receiver limited** (flow control kicks in)

Flow Control

- Determine the window size
  - Receiver continuously informs sender of amount of free buffer space
  - Uses: set *rcvr window size* field in TCP header
  - Sender: amount of transmitted unacked bytes is never larger than most recent *rcvr window size*

Silly Window Syndrome

- An application only reads one bit at a time…
  - What happens with the window? How much does it open?
    - Receiver sends message that window is 1 byte large
    - Sender sends packet with size 1 byte
    - Sender waits until window opens again

  ➔Throughput decreases drastically

  ➔Headers add additional unnecessary overhead

- Solutions
  - Only send Window update, when window of significant size
  - **Nagle Algorithm:** Do not send small segment, if small segment is already in flight

Ideal Window Size

- Window size has implications on throughput
  - $Data\ Rate\ r = \frac{Window\ Size\ w}{RTT}$
- Allows us to calculate optimal window for known *r* and RTT
  - $Ideal\ Size\ w^* = RTT\ \times Bandwidth\ B$
  - $Window\ Size\ w < RTT \times Bandwidth\ B$

    ➔ wasted bandwidth
  - $Window\ Size\ w > RTT\ \times Bandwidth\ B$

    ➔ Queuing at intermediate routers increased RTT

    ➔ Eventually packet loss
- Bandwidth-delay product $\mathrm{BDP} = RTT\ \times Bottleneck\ Bitrate$

Simulating Transport Protocols

- Network simulators
  - Allow evaluation of protocols
  - Allow simulating interactions between different protocols
- Examples
  - Network Simulator (NS)
  - SSFNet
  - Omnet++
  - ...
- Visualization of NS traces via NAM (Network Animator)